

IF29 — Data Analytics

Comparaison de deux méthodes de classification

Rapport de projet

ARIDORY Malcolm

BEN ABDALLAH Mohamed

LANOUAR Mariem

RODRIGUES Gwendal

UTT — Printemps 2026

Contents

1. Introduction	4
1.1. Contexte	4
1.2. Problématique	4
1.3. Définition d'un profil atypique	4
1.4. Structure du rapport	5
2. Description des données	6
2.1. Source et format	6
2.2. Volumétrie	6
2.3. Choix du système de stockage	6
3. Pipeline ETL (Extract - Transform - Load)	7
3.1. Architecture générale	7
3.2. Extraction : ingestion des tweets (<code>ingest.py</code>)	7
3.3. Transformation : déduplication des utilisateurs (<code>extract_users.py</code>)	7
3.4. Chargement : upsert vers la table <code>users</code> (<code>load_users.py</code>)	8
3.5. Indexation pour la performance	8
3.6. Structure finale de la table <code>users</code>	9
4. Labellisation manuelle des profils	10
4.1. Méthodologie d'échantillonnage	10
4.2. Fichier d'annotation	10
4.3. Critères de labellisation	11
4.3.1. Indicateurs de profil (User-based features)	11
4.3.2. Indicateurs d'activité (Activity-based features)	12
4.3.3. Indicateurs de contenu (Content-based features)	12
4.4. Processus de labellisation	12
5. Analyse exploratoire des données (EDA)	14
5.1. Distribution des classes	14
5.2. Analyse bivariée : Graphe social et Engagement	15
5.3. Personnalisation du profil	15
5.4. Matrice de corrélation	16
5.5. Synthèse dimensionnelle (Scores SPOT)	17
6. Méthodologie de modélisation	18
6.1. Feature engineering	18
6.2. Préparation des données pour l'apprentissage	19
6.2.1. Nettoyage et exclusions	19
6.2.2. Traitement de la colinéarité	20
6.2.3. Choix du type de normalisation	20
6.3. Modélisation Supervisée : Random Forest	20
6.3.1. Échantillonnage et validation	20
6.3.2. Optimisation des hyperparamètres	20
6.3.3. Évaluation et Matrice de Confusion	21
6.4. Modélisation Supervisée : SVM	22
7. Modélisation Non-Supervisée	23

8. Discussions et Conclusion	24
8.1. Modèle supervisé	24
8.2. Modèle non-supervisé	24
8.3. Comparaison des deux approches	24
9. Résultats	25
9.1. Performances du modèle supervisé	25
9.2. Résultats du clustering non-supervisé	25
9.3. Analyse comparative	25
10. Discussion, limites et perspectives	26
10.1. Interprétation des résultats	26
10.2. Limites	26
10.3. Perspectives d'industrialisation	26
11. Bibliographie	27

1. Introduction

1.1. Contexte

Twitter (aujourd'hui X) est l'une des principales plateformes de microblogging au monde, avec plusieurs centaines de millions d'utilisateurs actifs générant un flux continu de messages courts appelés **tweets**. Cette masse de données, accessible via des APIs publiques, constitue un terrain d'analyse privilégié pour les sciences des données, qu'il s'agisse d'étudier des phénomènes sociaux, politiques ou informationnels.

Cependant, tous les profils présents sur la plateforme ne sont pas animés par des humains. Une part significative de l'activité est générée par des comptes automatisés, appelés **bots**, dont les objectifs peuvent varier de la simple diffusion d'informations (bots météorologiques, alertes sismiques) à des usages malveillants (spam, phishing, manipulation de l'opinion, désinformation). La littérature scientifique s'est emparée de cette problématique depuis le milieu des années 2010, avec des travaux fondateurs comme ceux de Ferrara et al. (2016) sur la montée des bots sociaux, de Chu et al. (2012) sur la distinction entre humain, bot et cyborg, ou encore de Varol et al. (2017) sur la détection et la caractérisation des interactions humain-bot.

1.2. Problématique

Ce projet s'inscrit dans cette continuité. Il vise à répondre à la question suivante : **comment distinguer automatiquement un profil humain d'un profil atypique (bot, spammeur, compte automatisé) à partir des données publiques de Twitter ?**

Plus spécifiquement, nous cherchons à comparer deux paradigmes de classification :

- Une **approche supervisée** : entraîner un modèle sur un échantillon de profils préalablement labellisés manuellement (humain / bot).
- Une **approche non-supervisée** : laisser un algorithme de clustering identifier des groupes naturels dans les données, puis interpréter ces groupes **a posteriori**.

COMPLÉTER : une ou deux phrases sur le jeu de données World Cup et son intérêt spécifique (période, événement, volume).

1.3. Définition d'un profil atypique

Dans le cadre de ce projet, nous définissons un **profil atypique** comme un compte dont les caractéristiques comportementales et statistiques s'écartent significativement de celles de la population moyenne des utilisateurs. Cette définition, volontairement large, recouvre plusieurs réalités :

- **Les bots** : comptes entièrement ou partiellement automatisés, caractérisés par une fréquence de publication anormalement élevée, un ratio followers/followees déséquilibré, et une faible personnalisation du profil (Ferrara et al., 2016 ; Chu et al., 2012).
- **Les spammeurs** : comptes diffusant massivement des URLs, souvent raccourcies, dans un but publicitaire ou malveillant (Benevenuto et al., 2010).
- **Les profils de désinformation** : comptes créés pour amplifier artificiellement certains messages ou simuler un soutien populaire (astroturfing).

COMPLÉTER : précisez si vous vous concentrez uniquement sur les bots ou si vous incluez d'autres catégories. Justifiez votre choix au regard du jeu de données et des références citées.

1.4. Structure du rapport

Ce rapport suit le déroulement classique d'un projet de data science, en reprenant avec souplesse une approche semblable à celle du framework CRISP-DM : nous avons la compréhension métier, nous poursuivrons avec l'analyse des données, la préparation de notre base de données et la pipeline ETL, les modélisations (supervisée et non-supervisée) et enfin l'évaluation et la mise en production de nos modèles.

2. Description des données

2.1. Source et format

Le jeu de données **Tweet_Worldcup.zip** est constitué de fichiers JSON contenant des tweets collectés pendant la Coupe du Monde de football 2018. Chaque ligne de chaque fichier correspond à un tweet au format JSON tel que renvoyé par l'API Twitter v1.

Un tweet typique contient :

- Les **métadonnées du tweet** : identifiant (**id**), date de création (**created_at**), texte (**full_text** ou **text**), langue (**lang**).
- Les **entités** mentionnées dans le tweet : hashtags (**#**), mentions (**@**), URLs, médias attachés.
- L'**objet utilisateur (user object)** imbriqué : identifiant (**user.id**), pseudonyme (**screen_name**), nom affiché (**name**), description, nombre de followers, nombre d'amis (**following**), nombre de statuts postés, statut de vérification, date de création du compte, etc.

Le fait que l'objet utilisateur soit **imbriqué** dans chaque tweet est une caractéristique importante du schéma de données : un même utilisateur apparaît autant de fois qu'il a de tweets dans le corpus, avec potentiellement des valeurs différentes de followers, friends, etc. selon la date du tweet (instantané du profil à l'instant de la collecte).

2.2. Volumétrie

INSÉRER ICI LES STATISTIQUES DE VOLUMÉTRIE : Nombre total de tweets dans la base, nombre total d'utilisateurs distincts, période couverte (premier et dernier tweet), taille de la base de données. Vous pouvez obtenir ces chiffres avec les requêtes SQL correspondantes.

À titre indicatif, après ingestion complète, notre base de données contient environ **4,5 millions de tweets** émis par plus de **1,8 million d'utilisateurs distincts**.

2.3. Choix du système de stockage

Le sujet du projet suggère MongoDB pour le stockage des données semi-structurées. Nous avons fait le choix alternatif de **PostgreSQL** pour les raisons suivantes : la **maturité de l'écosystème SQL** pour l'agrégation et l'analyse exploratoire (fenêtrage, agrégats, jointures) ; le support natif du type **JSONB**, qui permet de conserver les données brutes dans leur format d'origine tout en offrant des capacités d'indexation et de requêtage performantes via les expressions de chemin JSON ; et la **compatibilité avec les outils Python** de data science (psycopg2, pandas).

Ce choix constitue un compromis entre la flexibilité du NoSQL et la puissance du relationnel, particulièrement adapté à un projet mêlant données brutes (JSONB) et données structurées (tables relationnelles).

3. Pipeline ETL (Extract - Transform - Load)

Cette section décrit l'architecture du pipeline de traitement des données, depuis l'ingestion des fichiers JSON bruts jusqu'à la constitution d'une table `users` exploitable pour la classification.

3.1. Architecture générale

Le pipeline s'articule autour de deux tables PostgreSQL :

- `tweets` : stocke les tweets bruts dans leur intégralité au format JSONB. Un index d'expression est créé sur l'identifiant utilisateur extrait du JSON pour permettre des requêtes performantes par utilisateur.
- `users` : table relationnelle contenant un enregistrement par utilisateur distinct, avec le snapshot utilisateur le plus récent (dernier tweet connu).

INSÉRER ICI UN SCHÉMA DU PIPELINE : fichiers JSON → `ingest.py` → `tweets` → `extract_users.py` → `users`. Un diagramme simple fait avec Draw.io ou Excalidraw suffit.

3.2. Extraction : ingestion des tweets (`ingest.py`)

Le script `src/etl/extract/ingest.py` est responsable du chargement initial des fichiers JSON dans la table `tweets`. Son fonctionnement est le suivant :

1. Il lit récursivement tous les fichiers `.json` du répertoire `data/raw/`.
2. Pour chaque fichier, chaque ligne est parsée comme un objet JSON.
3. Les champs `id`, `created_at` et `text` (ou `full_text`) sont extraits et stockés dans des colonnes SQL typées ; l'intégralité du JSON est conservée dans la colonne `raw_data` de type JSONB.
4. L'insertion se fait par lots (`batch`) de 2 000 tweets via `execute_values` de `psycopg2`, avec une clause `ON CONFLICT (id) DO NOTHING` pour garantir l'idempotence (un fichier déjà ingéré ne duplique pas les données).
5. Un fichier `processed_files.log` assure le suivi des fichiers déjà traités, permettant de reprendre l'ingestion après une interruption sans repartir de zéro.

COMPLÉTER : indiquez le temps d'exécution total de l'ingestion ainsi que les éventuels problèmes rencontrés (lignes mal formatées, erreurs de parsing).

La structure de la table `tweets` est la suivante :

Colonne	Type	Description
<code>id</code>	BIGINT	Identifiant unique du tweet (PK)
<code>created_at</code>	TIMESTAMP	Date de création du tweet
<code>tweet_text</code>	TEXT	Texte du tweet
<code>raw_data</code>	JSONB	Objet JSON complet du tweet

3.3. Transformation : déduplication des utilisateurs (`extract_users.py`)

Le cœur de la transformation réside dans le script `src/etl/transform/extract_users.py`.

Problème. La table `tweets` peut contenir plusieurs occurrences du même utilisateur, chacune avec un snapshot potentiellement différent (le nombre de followers évolue dans le temps). Il est nécessaire de choisir quelle version retenir.

Solution retenue : snapshot le plus récent. Nous utilisons une clause `DISTINCT ON` de PostgreSQL triée par `user_id`, `created_at` DESC. Cela garantit que, pour chaque utilisateur, seul

le tweet le plus récent est conservé — et donc la version la plus à jour de son profil **au moment de la collecte**. Cette approche est préférable à une moyenne (le nombre de followers n'est pas une mesure qu'on lisse) et à un snapshot aléatoire (non déterministe).

La requête SQL sous-jacente est :

```
SELECT DISTINCT ON ((raw_data->'user'->>'id')::BIGINT)
    (raw_data->'user'->>'id')::BIGINT AS user_id,
    raw_data->'user' AS raw_user,
    created_at AS tweet_created_at
FROM tweets
WHERE raw_data->'user'->>'id' IS NOT NULL
ORDER BY (raw_data->'user'->>'id')::BIGINT, created_at DESC
```

Une deuxième passe agrège par utilisateur les statistiques `tweet_count`, `first_seen_at` et `last_seen_at` (premier et dernier tweet dans notre corpus).

COMPLÉTER : indiquez le temps d'exécution de l'extraction des utilisateurs (environ 20-25 minutes pour 4,5M de tweets). Mentionnez l'utilisation du curseur côté serveur (`server-side cursor`) pour éviter de charger toutes les données en mémoire.

3.4. Chargement : upsert vers la table `users` (`load_users.py`)

Le script `src/etl/load/load_users.py` orchestre l'ensemble du pipeline :

1. Il applique `init.sql` (création de la table `users` et des index d'expression) si ce n'est pas déjà fait.
2. Il appelle `extract_users()` pour obtenir la liste des utilisateurs uniques.
3. Il insère les utilisateurs dans la table `users` par lots de 50 000, avec une clause `ON CONFLICT (user_id) DO UPDATE` (upsert) qui garantit l'idempotence.

Un mécanisme de **checkpoint** (fichier pickle) sauvegarde la liste extraite avant l'upsert. En cas d'échec, le prochain lancement recharge le pickle et saute l'extraction, évitant ainsi de perdre 25 minutes de calcul.

COMPLÉTER : indiquez le temps d'exécution total du chargement, le nombre d'utilisateurs chargés, et mentionnez les difficultés rencontrées (volume de données, sérialisation JSONB, transactions PostgreSQL).

3.5. Indexation pour la performance

Pour éviter des parcours complets de la table `tweets` à chaque requête par utilisateur, nous avons créé deux index d'expression :

```
CREATE INDEX idx_tweets_user_id
ON tweets (((raw_data->'user'->>'id')::BIGINT)));

CREATE INDEX idx_tweets_user_date
ON tweets (((raw_data->'user'->>'id')::BIGINT), created_at DESC);
```

Ces index permettent à PostgreSQL d'utiliser des parcours d'index (index scan) plutôt que des parcours séquentiels (sequential scan) pour les requêtes filtrant ou triant par utilisateur. Le choix d'un index d'expression plutôt que d'une colonne générée (`GENERATED ALWAYS AS ... STORED`) est motivé par le souci de ne pas réécrire la table `tweets` qui pèse plusieurs dizaines de gigaoctets.

3.6. Structure finale de la table users

Colonne	Type	Description
user_id	BIGINT	Identifiant Twitter du compte (PK)
screen_name	TEXT	Pseudonyme
name	TEXT	Nom affiché
description	TEXT	Bio du profil
location	TEXT	Localisation déclarée
url	TEXT	URL dans le profil
followers_count	INT	Nombre d'abonnés
friends_count	INT	Nombre d'abonnements (following)
listed_count	INT	Nombre de listes incluant ce compte
favourites_count	INT	Nombre de likes émis
statuses_count	INT	Nombre total de tweets postés
verified	BOOLEAN	Compte certifié par Twitter
created_at	TIMESTAMP	Date de création du compte
profile_image_url	TEXT	URL de la photo de profil
default_profile	BOOLEAN	Profil jamais personnalisé
default_profile_image	BOOLEAN	Photo de profil par défaut
raw_user	JSONB	Objet utilisateur complet (JSON)
tweet_count	INT	Tweets de cet utilisateur dans le corpus
last_tweet_at	TIMESTAMP	Date du tweet le plus récent
first_seen_at	TIMESTAMP	Date du premier tweet capturé
last_seen_at	TIMESTAMP	Date du dernier tweet capturé

4. Labellisation manuelle des profils

L'approche supervisée de classification nécessite un jeu de données d'entraînement étiqueté. En l'absence de **ground truth** (les données Twitter ne contiennent pas de label « bot » ou « humain »), nous avons constitué manuellement un corpus annoté.

4.1. Méthodologie d'échantillonnage

Le script `src/ml/sample_users.py` extrait un échantillon aléatoire de 1 000 utilisateurs depuis la table `users`. La sélection est **déterministe et reproductible** : elle utilise un hachage MD5 combinant l'identifiant utilisateur et une graine fixe (*seed*), ce qui garantit que la même graine produit toujours le même échantillon, indépendamment de la version de PostgreSQL ou de l'état du générateur aléatoire :

```
SELECT user_id FROM users
ORDER BY MD5(user_id::text || 'if29')
LIMIT 1000
```

4.2. Fichier d'annotation

Pour chaque utilisateur, le CSV exporté contient : les **métadonnées du profil** (20 colonnes issues de la table `users`) ; des **features calculées** aidant à la décision (ratio followers/friends, tweets par jour depuis la création du compte, tweets par jour dans le corpus, durée de couverture dans le corpus) ; les **5 tweets les plus récents** de l'utilisateur dans notre corpus, concaténés dans une colonne `sample_tweets` ; et deux colonnes vides à remplir : `label` (0 = humain, 1 = bot) et `notes`.

Le tableau ci-dessous détaille les indicateurs clés mis à disposition de l'annotateur.

Colonne	Signification	Utilisation
<code>followers_count</code>	Nombre d'abonnés	Un compte avec 0 followers et beaucoup de tweets est suspect
<code>friends_count</code>	Abonnements (following)	Un ratio followers/friends très déséquilibré (>5) suggère un bot
<code>listed_count</code>	Listes incluant le compte	Un compte jamais listé (<2) est peu visible, potentiellement automatisé
<code>statuses_count</code>	Tweets totaux postés	Un compte récent avec >10k statuts indique une automatisation
<code>favourites_count</code>	Likes émis	Un bot like souvent mécaniquement ; un humain a un ratio likes/tweets plausible
<code>verified</code>	Compte certifié (blue check)	Un compte vérifié est quasiment certainement humain (ou organisation légitime)

Colonne	Signification	Utilisation
account_created_at	Date de création du compte	Un compte créé récemment mais déjà très actif est suspect
default_profile	Profil jamais personnalisé	Laissé à true = aucune customisation = indice de bot
default_profile_image	Photo par défaut (oeuf)	Idem, absence d'avatar personnalisé
followers_friends_ratio	Ratio friends/followers	>1 = suit plus qu'il n'est suivi ; >5 = fort soupçon de bot
tweets_per_day_since_creation	Tweets/jour depuis création	>50 tweets/jour en moyenne est humainement impossible en continu
dataset_coverage_days	Jours de présence dans le corpus	Permet de contextualiser le tweet_count
tweets_per_day_in_dataset	Tweets/jour dans le corpus	Même interprétation que ci-dessus, limité à notre fenêtre d'observation
sample_tweets	5 derniers tweets	Lecture du contenu : le texte est-il varié ? naturel ? dupliqué ?

4.3. Critères de labellisation

Nous avons adopté une grille d'analyse fondée sur la littérature scientifique, en nous appuyant notamment sur les travaux de Ferrara et al. (2016), Chu et al. (2012) et Varol et al. (2017), ainsi que sur le framework SPOT de Perez et al. (2011). Les critères sont classés en trois catégories.

4.3.1. Indicateurs de profil (User-based features)

Ces indicateurs portent sur les métadonnées du compte, indépendamment de son activité récente. Un profil atypique présente typiquement :

- Une **absence de personnalisation** : `default_profile = true` et `default_profile_image = true` simultanément signalent un compte généré automatiquement sans effort de personnalisation humaine.
- Un **déséquilibre du graphe social** : un ratio `friends / followers` supérieur à 5 indique un compte qui suit massivement sans être suivi en retour — comportement caractéristique des bots de type « follow spam ».
- Une **date de création récente** couplée à un volume de statuts anormalement élevé (ex. compte créé il y a 3 mois avec 50 000 tweets).

4.3.2. Indicateurs d'activité (Activity-based features)

Ces indicateurs sont calculés à partir de l'activité observée du compte :

- Une **fréquence de publication excessive** : un rythme soutenu de plus de 50 tweets par jour, maintenu sur une longue période, dépasse les capacités humaines. Le papier SPOT (Perez et al., 2011) qualifie ce phénomène de **degré d'agressivité**.
- Une **présence massive d'URLs** dans les tweets : les bots diffusent souvent des liens raccourcis à des fins de spam ou de phishing.

4.3.3. Indicateurs de contenu (Content-based features)

L'analyse du texte des tweets (`sample_tweets`) permet de trancher les cas ambigus :

- **Duplication de contenu** : des tweets quasi-identiques postés à intervalles réguliers indiquent une automation par template.
- **Cohérence sémantique** : un humain produit des tweets variés, contextuels, parfois avec des fautes de frappe, des émotions, des réactions à d'autres utilisateurs.
- **Présence de hashtags et mentions** : un usage excessif et systématique de # et @ dans chaque tweet est un marqueur d'automation (visibilité optimisée dans le cadre SPOT).

4.4. Processus de labellisation

Afin de labelliser les différents profils, nous avons ouvert le fichier CSV dans un tableur, et nous avons déterminé par inspection visuelle, en prenant en compte les différentes métriques calculées, si le profil que nous avons sous les yeux nous semblait être celui d'un bot ou non.

	followers_count	friends_count	listed_count	favourites_count	statuses_count	verified	account_created_at	default_profile	default_profile_image	tweet_count_in_dataset	first_seen_at	last_seen_at	followers_friends_ratio	tweets_per_day_since_creation
1	553	1564	0	5293	2420	False	2011-06-16T13:10:52	True	False	18	2018-06-15T10:53:35	2018-06-17T15:41:55	2.8282	0.85
2	697	611	8	24023	37893	False	2013-12-17T15:13:28	False	False	26	2018-06-14T07:50:34	2018-06-17T11:33:59	0.8766	23.08
3	492	628	64	39238	210310	False	2013-05-31T13:57:29	True	False	17	2018-06-14T10:09:42	2018-06-16T14:57:29	1.2764	114.17
4	5177	3018	7	5064	19229	False	2011-06-14T09:44:55	False	False	8	2018-06-14T17:19:23	2018-06-16T15:18:26	0.583	7.51
5	2749	2149	14	7382	9021	False	2013-03-18T09:32:36	False	False	9	2018-06-15T12:22:35	2018-06-16T20:27:35	0.7817	4.71
6	133	218	5	3693	6003	False	2014-10-19T07:44:57	False	False	8	2018-06-14T03:12:48	2018-06-16T01:32:54	1.6391	4.5
7	11576	436	100	168490	79585	False	2012-01-12T12:59:23	False	False	22	2018-06-16T13:32:33	2018-06-17T16:43:48	0.0377	33.89
8	1233	1831	9	5118	72754	False	2012-09-28T09:44:21	False	False	22	2018-06-14T02:38:24	2018-06-17T14:21:01	1.485	34.84
9	595	165	37	53216	31186	False	2014-11-30T17:48:40	False	False	22	2018-06-14T09:57:54	2018-06-17T11:40:28	0.2773	24.1
10	1009	4887	1	12036	15518	False	2012-01-22T19:50:34	True	False	13	2018-06-14T08:05:42	2018-06-16T16:10:57	4.8434	6.84
11	242	432	9	2409	14460	False	2011-05-28T21:27:38	True	False	19	2018-06-14T08:12:03	2018-06-17T15:10:09	1.7851	5.81
12	994	960	12	3796	28021	False	2009-07-31T20:08:13	False	False	6	2018-06-14T18:31:29	2018-06-15T18:30:16	0.9658	8.65
13	1321	206	9	648	16259	False	2011-09-18T02:25:19	True	False	6	2018-06-15T20:08:49	2018-06-16T21:09:32	0.1559	6.6
14														
15	397	371	11	1293	107103	False	2012-01-31T04:31:49	False	False	12	2018-06-15T12:24:52	2018-06-17T16:50:49	0.8345	45.99
16	5970	2000	184	4856	13900	False	2009-04-08T12:46:57	False	False	6	2018-06-15T14:22:02	2018-06-17T07:25:43	0.335	4.14
17	539	2226	15	57079	31875	False	2013-08-03T01:27:43	False	False	6	2018-06-14T03:59:07	2018-06-16T13:49:53	4.1299	17.83
18	192	266	3	1555	5753	False	2009-08-12T12:22:57	True	False	12	2018-06-14T13:58:16	2018-06-16T18:42:48	1.3654	1.78
19	390	550	14	730	22393	False	2009-05-27T15:57:58	False	False	11	2018-06-15T13:42:43	2018-06-17T15:42:16	1.4103	6.77

Deux membres du groupe ont labellisé ensemble, ligne par ligne, environ 200 profils. Une astuce a été de filtrer et trier pour voir en priorité les profils qui correspondaient à des critères suspects et essayer de trancher les cas ambigus. Certains profils apparaissent assez rapidement comme humains ou comme suspects, ceux-ci ont été rapidement labellisés. Nous avons ensuite demandé à un LLM de confirmer ou infirmer nos raisonnements (qu'il a validés sauf pour deux cas que nous avons tranchés), et avons conjointement classé 200 profils supplémentaires, par paquets

de 50 où nous avons inversé les rôles : cette fois-ci, nous étions assignés à la deuxième lecture. Au total, notre échantillon final s'élève à 420 profils labellisés (après nettoyage), ce qui semble être un volume correct pour la mise en place de notre algorithme supervisé. Évidemment, un tel algorithme est plus performant sur de grandes quantités d'observations, mais il n'est pas humainement imaginable de labelliser les 1,8 million de profils, et nous avons souhaité éviter toute sorte d'interpolation pour ne pas induire de biais.

5. Analyse exploratoire des données (EDA)

Avant d'entamer la phase de modélisation, nous avons réalisé une analyse exploratoire sur notre échantillon final de 420 profils labellisés. Cette étape permet de valider nos hypothèses sur les caractéristiques discriminantes entre bots et humains, et d'orienter nos choix d'ingénierie des features.

5.1. Distribution des classes

Après nettoyage des valeurs nulles, notre base annotée compte **420 profils**, répartis de la manière suivante :

- **348 profils labellisés « Humain »** (env. 83 %)
- **72 profils labellisés « Bot »** (env. 17 %)

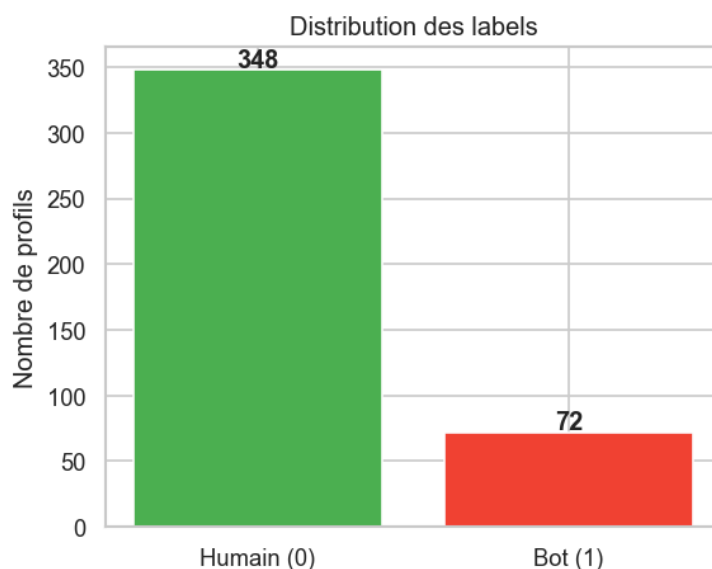


Figure 1: Distribution des labels dans l'échantillon annoté

Ce net déséquilibre naturel en faveur des comptes humains est classique sur les réseaux sociaux. Il implique qu'il faudra redoubler de vigilance lors de l'entraînement de nos modèles supervisés (stratification, pondération des classes) et dans le choix de nos métriques d'évaluation (F1-score, Precision-Recall AUC plutôt que la simple accuracy).

5.2. Analyse bivariée : Graphe social et Engagement

L'étude des distributions bivariées met en évidence des comportements distincts. Par exemple, si l'on observe la relation entre le nombre d'abonnés et le nombre d'abonnements (en échelle logarithmique), on remarque que les bots ont tendance à suivre beaucoup de comptes pour un nombre d'abonnés nettement inférieur. À l'inverse, les humains suivent une tendance plus linéaire et proportionnelle.

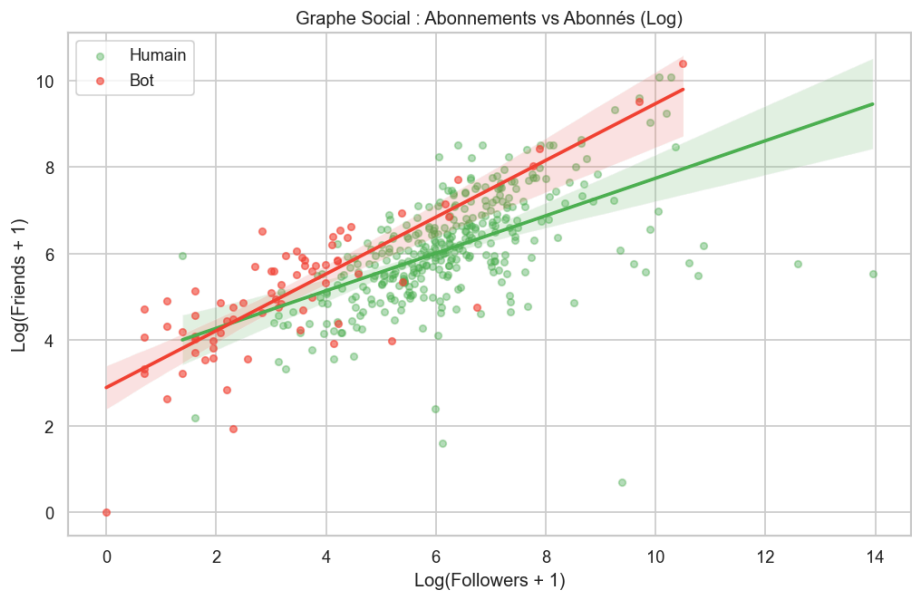


Figure 2: Graphe Social : Abonnements vs Abonnés (Log)

5.3. Personnalisation du profil

L'absence de personnalisation est un signal fort d'automatisation. Nos données le confirment de manière frappante : plus de 90 % des comptes humains de l'échantillon disposent d'une description (bio) complétée, tandis que près de la moitié des bots se contentent d'un profil vide à ce niveau.

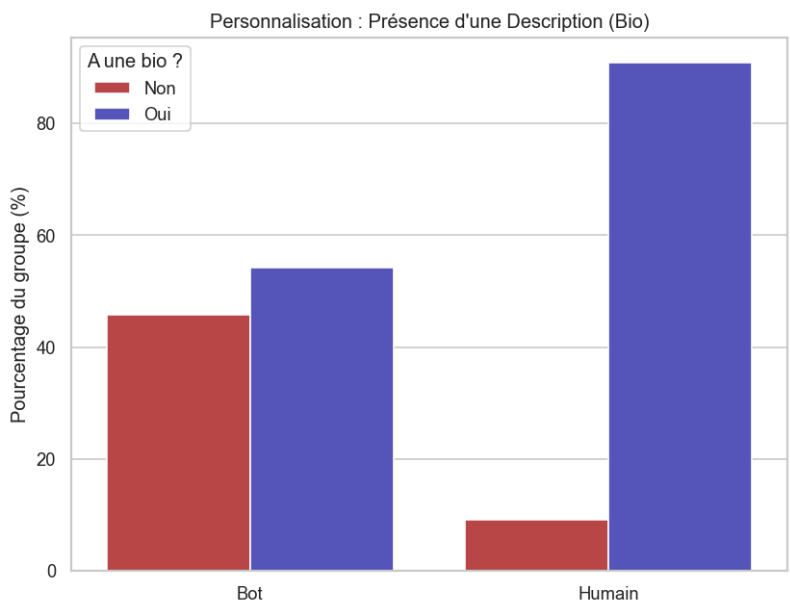


Figure 3: Présence d'une description selon la classe

5.4. Matrice de corrélation

L'analyse de la matrice de corrélation sur notre échantillon met en évidence plusieurs associations linéaires intéressantes qui viennent conforter nos hypothèses :

- **Influence et notoriété** : On observe une forte corrélation positive croisée entre le nombre d'abonnés (`followers_count`), d'apparitions dans des listes (`listed_count`) et le statut vérifié (`verified`). Les comptes suivis sont logiquement plus souvent répertoriés et certifiés.
- **Engagement** : Le volume de publications (`statuses_count`) est corrélé positivement au nombre de favoris émis (`favourites_count`). Les profils actifs dans la création de contenu le sont également dans l'interaction.
- **Ancienneté et personnalisation** : L'âge du compte (`account_age_days`) est corrélé négativement avec le maintien d'un profil par défaut (`default_profile`) : les vieux comptes ont presque tous fini par modifier leur profil au fil du temps. On constate aussi que l'absence de description dans la bio a tendance à coïncider avec un plus petit ratio friends/followers.
- **Cible (label)** : Bien qu'il s'agisse de corrélations point à point qui ne capturent pas les relations non-linéaires plus complexes, le label "bot" est positivement lié au ratio `followers_friends_ratio` (suivi massif sans retour) et à la densité de publications (`tweets_per_day_since_creation`), ce qui valide pleinement l'inclusion de ces métriques clés.

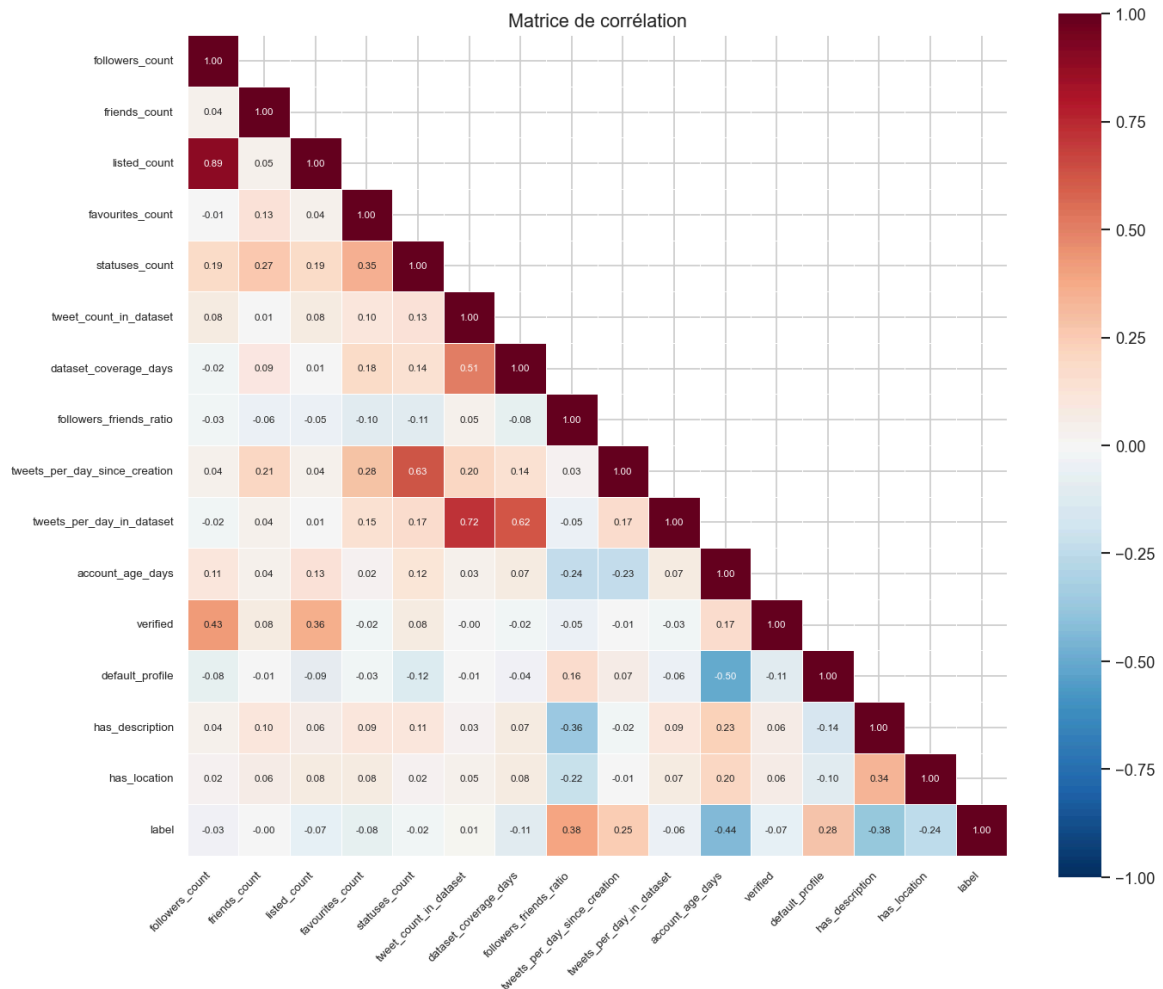


Figure 4: Matrice de corrélation des features

5.5. Synthèse dimensionnelle (Scores SPOT)

Nous avons également examiné la pertinence des dimensions extraites avec l'approche SPOT (Agressivité, Visibilité, Danger). Les boîtes à moustaches confirment la pertinence de certains de ces indicateurs :

- **L'agressivité** (fréquence combinée de tweets et d'ajouts d'amis) présente nettement plus de valeurs extrêmes chez les bots.
- **Le danger** (proportion de tweets contenant des URLs) est, en médiane, assez peu discriminant ici, mais on observe un étalement important des valeurs extrêmes chez les deux groupes.

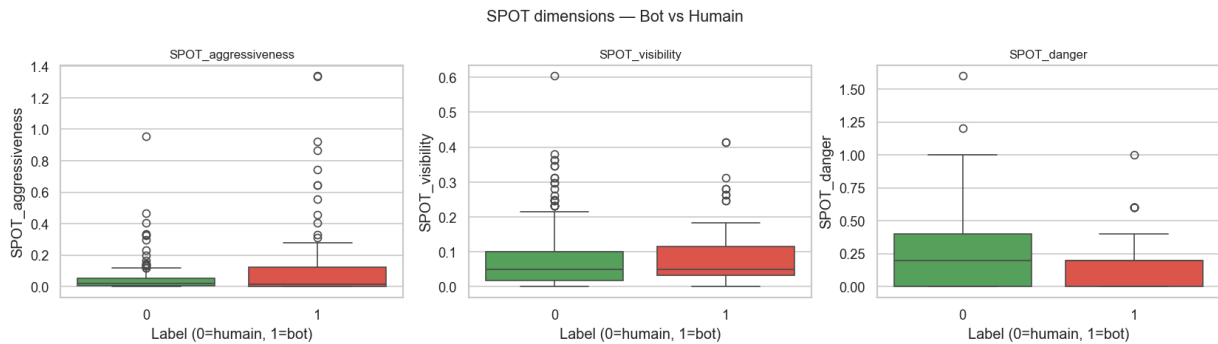


Figure 5: Dimensions SPOT évaluées sur notre échantillon : Agressivité, Visibilité, Danger

Ces constats exploratoires nous confortent dans le choix de ces métriques comme variables d'entrée pour nos algorithmes de modélisation.

6. Méthodologie de modélisation

6.1. Feature engineering

Nous avons commencé par calculer toutes les métriques utiles dans une `materialized view` en base de données, notamment celles repérées pour l'annotation : ratio friends/followers, âge du compte, fréquences de publication, agressivité SPOT, etc. À celles-ci s'ajoute une analyse des sources : le payload de l'API twitter donne un champ `source` qui est un lien html (tag `<a>`) duquel on peut extraire un nom. Nous avons extrait cette information avec la requête suivante :

```
SELECT
    COALESCE(
        NULLIF(trim(substring(raw_data->>'source' from '<a[^>]*>([<*>]*)</a>'), ''),
        raw_data->>'source'
    ) AS tweet_source,
    COUNT(*) AS tweet_count
FROM
    tweets
GROUP BY
    1
ORDER BY
    2 DESC
;
```

À partir de là, on obtient une liste de plusieurs centaines de valeurs :

tweet_source	tweet_count
Twitter for Android	1879304
Twitter for iPhone	1660451
Twitter Web Client	460123
Twitter Lite	147823
TweetDeck	72768
Twitter for iPad	60325
Mobile Web (M2)	18292
IFTTT	13063
Facebook	12681
Paper.li	12206
Tweetbot for iOS	10523
Hootsuite Inc.	9002
FCBarcelonaBot	6857
...	

Nous avons donné cette liste à un LLM (Moonshot AI Kimi K2.6) afin qu'il identifie et cherche les sources connues comme étant des outils communs d'automatisation ou des bots. Cette approche est approximative : les bots plus sophistiqués peuvent parfois tenter de maquiller leur origine. En revanche, les bots étant constants dans leurs sources, nous avons également utilisé la variété des sources comme feature pour essayer de déterminer les bots, car un humain, même s'il utilise principalement une source, a souvent l'occasion d'en utiliser d'autres s'il a plusieurs appareils (en général : Client Web, iOS, Android, iPadOS...)

Afin d'entraîner nos modèles, nous avons structuré nos attributs (**features**) en trois grandes catégories, en nous inspirant de la littérature existante sur la détection d'anomalies sur les réseaux sociaux.

- **Caractéristiques liées au profil (User-based)** : Nous prenons en compte la présence d'éléments descriptifs (`has_description`, `has_url`, `has_location`, `description_length`), la structure du pseudonyme (`screen_name_length`, `screen_name_has_digits`), ainsi que des mesures d'influence et de socialisation telles que la réputation (ratio $\frac{\text{followers}}{\text{followers} + \text{friends} + 1}$) ou le ciblage des listes (`listed_per_follower`). Le ratio **followers/friends** est une métrique classique pour identifier les bots qui suivent massivement d'autres comptes sans être suivis en retour (Chu et al., 2012). L'ancienneté du compte (`account_age_days`) est également pertinente, les vagues de création massive de faux comptes étant fréquentes.
- **Caractéristiques d'activité (Activity-based)** : L'intensité de publication est évaluée via le volume total de tweets (`tweet_count`), la fréquence historique depuis la création (`tweet_frequency`), ou encore la densité d'activité calculée pendant la période de collecte (`tweets_per_day_in_dataset`). Ces mesures traduisent le comportement intensif caractéristique de nombreux comptes automatisés. L'intérêt pour les interactions est évalué à travers le taux de favoris donnés (`favourites_per_status`). Le délai depuis le dernier tweet (`days_since_last_tweet`) ou l'étendue de l'observation (`observation_span_days`) donnent une mesure de l'assiduité du compte.
- **Caractéristiques de contenu (Content-based)** : Ces attributs caractérisent la nature des messages émis. Nous calculons la fréquence d'utilisation d'URLs, de hashtags, et de mentions (`urls_per_tweet`, `hashtags_per_tweet`, `mentions_per_tweet`), ces derniers étant souvent saturés par les bots de spam à visée promotionnelle. Enfin, l'orientation vers le relais d'information (au lieu de la production originale) est mesurée par la proportion de retweets (`retweet_rate`) et le taux de réponses (`reply_rate`). Nous y ajoutons bien sûr le taux de sources suspectes ou automatisées (`bot_source_ratio`) et la diversité des outils employés (`unique_sources`) évoqués plus haut.

Le script SQL complet disponible sur GitHub.

6.2. Préparation des données pour l'apprentissage

Avant d'injecter nos données dans les modèles, plusieurs transformations ont été appliquées pour garantir leur exploitabilité.

6.2.1. Nettoyage et exclusions

Nous avons supprimé du jeu d'entraînement les identifiants explicites (`user_id`, `screen_name`, `name`), ainsi que les URL de profil et descriptions sous format texte, le modèle n'étant conçu que pour traiter des données numériques. Les dates de création et d'activité (`created_at`, `last_tweet_at`, etc.) ont également été écartées, l'essentiel de l'information temporelle étant déjà capturée et structurée sous forme de features (âge du compte, jours depuis le dernier tweet). Enfin, les valeurs booléennes (`has_location`, `has_url`...) ont été converties au format binaire entier, et les valeurs nulles (souvent causées par une absence de tweets permettant de calculer les ratios) ont été remplacées par 0.

6.2.2. Traitement de la colinéarité

Une caractéristique frappante de l'ingénierie des caractéristiques est l'apparition de forte redondance entre certains champs. Notre matrice de corrélation exploratoire l'avait révélé : des indicateurs tels que `followers_count` et `listed_count` peuvent être extrêmement liés. Bien qu'un modèle à base d'arbres puisse survivre à cette redondance (il sélectionnera les meilleures partitions de façon stochastique), une forte colinéarité nuit à l'interprétabilité globale (cf. *feature importance*) et alourdit l'entraînement. Malgré cela, il n'y a aucune corrélation qui soit supérieure à 90%

6.2.3. Choix du type de normalisation

L'approche de mise à l'échelle (**scaling**) n'est pas universelle. Dans ce projet impliquant une comparaison entre deux familles d'algorithmes fondamentalement différentes :

- Pour le modèle de **Forêts Aléatoires (Random Forest)**, aucune standardisation (`StandardScaler`) n'a été appliquée. Les algorithmes d'arbres partitionnent l'espace de manière orthogonale en divisant les fonctionnalités sur des seuils de décision (ex: $X > 100$). Une transformation monotone ou affine ne change pas la structure des nœuds ; elle est complètement invariante à l'échelle.
- À l'inverse, l'approche par **Machine à Vecteurs de Support (SVM)**, explorée par la suite, ainsi que le **Clustering**, reposent formellement sur le calcul matriciel de distances euclidiennes. Sur ces derniers, un espace de variables déséquilibré fausse l'hyperplan optimal. Il requiert un processus de standardisation systématique des caractéristiques, voire parfois une transformation logarithmique (**log1p**) sur les données comportant de grandes disproportions et d'importantes asymétries de distribution statistiques (**heavy-tailed data**).

6.3. Modélisation Supervisée : Random Forest

Pour expérimenter la classification supervisée, nous avons retenu l'algorithme **Random Forest** (Forêts Aléatoires). Ce modèle ensembliste repose sur la construction d'un grand nombre d'arbres de décision appliqués sur des sous-échantillons aléatoires. Il est naturellement robuste face au sur-apprentissage et accommode sans heurt un ensemble hétéroclite de features.

6.3.1. Échantillonnage et validation

Compte tenu du net déséquilibre de nos classes (17% de bots pour 83% d'humains), nous devons éviter d'avoir par hasard une représentation nulle des bots dans nos ensembles de validation. Le jeu de données a été scindé à l'aide de la fonction `train_test_split` avec les proportions 80% (Entraînement - `X_train`) et 20% (Test - `X_test`), en spécifiant un argument de **stratification**. Cette précaution garantit mathématiquement que la distribution initiale des classes est honorée dans chacun des échantillons.

6.3.2. Optimisation des hyperparamètres

Afin d'ajuster finement les performances du Random Forest, nous avons mis en place une exploration des hyperparamètres via une recherche sur grille exhaustive (`GridSearchCV`).

Les paramètres de cette grille comprenaient notamment :

- Le nombre d'arbres (`n_estimators`).
- L'architecture de la profondeur (`max_depth`).
- La flexibilité fine des feuillages terminaux (`min_samples_split`, `min_samples_leaf`).

- La pondération de classes (`class_weight = "balanced"`) face à l'asymétrie Humains/Bots.

L'optimisation a été pilotée en validation croisée stratifiée sur le sous-ensemble d'entraînement à 5 passes (`StratifiedKFold`, `n_splits=5`), employant, non pas la précision (**Accuracy**), mais spécifiquement le **F1-score** comme unique boussole d'apprentissage.

6.3.3. Évaluation et Matrice de Confusion

L'évaluation finale de notre modèle s'est faite sur l'ensemble de Test (20% des données), laissé jusque-là parfaitement invisible à l'apprentissage. Afin d'appréhender toute la granularité de ces résultats, particulièrement dans un contexte déséquilibré, nous nous sommes appuyés sur la matrice de confusion usuelle ainsi que sur plusieurs métriques, définies mathématiquement par les taux de Vrais Positifs (V_p), Faux Positifs (F_p), Vrais Négatifs (V_n) et Faux Négatifs (F_n) :

- **L'exactitude (Accuracy)** : ratio de prédictions correctes globales : $\frac{V_p + V_n}{\text{Total}}$. Bien qu'élevée à 0.917 (soit 91.7% de bonnes intuitions globales), elle peut s'avérer trompeuse en asymétrie de classes (il "suffit" d'étiqueter systématiquement la classe majoritaire pour obtenir un score honorable).
- **La précision (Precision)** : $\frac{V_p}{V_p + F_p}$. Parmi l'ensemble des comptes signalés comme "Bots" par notre modèle, quelle est la proportion réelle de vrais bots ? Elle s'élève ici à 0.684.
- **Le rappel (Recall ou Sensibilité)** : $\frac{V_p}{V_p + F_n}$. Parmi tous les bots **existant réellement** dans l'échantillon de test, combien le modèle a-t-il pu en identifier ? Avec notre configuration, cette métrique culmine à 0.929 (près de 93% des bots sont débusqués).
- **Le F1-Score** : la moyenne harmonique de la précision et du rappel, calculée selon $2 \cdot \frac{P \cdot R}{P + R}$. C'est le juge de paix. Face au déséquilibre de classe, notre modèle a abouti à un très bon score certifié par validation croisée de 0.788.

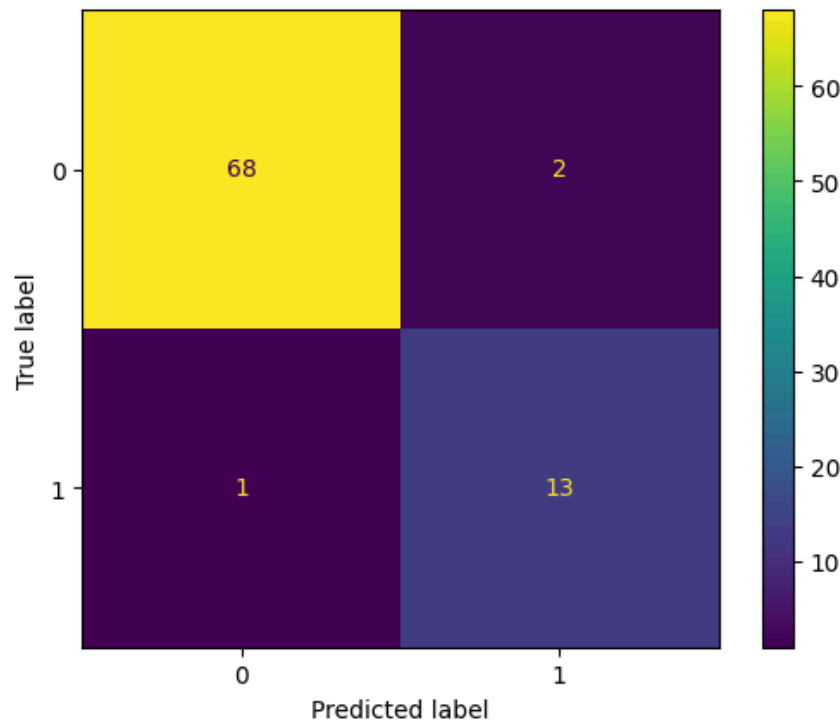


Figure 6: Matrice de confusion du modèle Random Forest sur l'ensemble de Test

Cette matrice, et en particulier l'excellent score de rappel, illustre le compromis de “paranoïa utile” atteint par le modèle lors de la recherche des hyperparamètres (le poids **balanced** étant retenu). Il s'avère doté d'une redoutable capacité de détection des profils bots isolés, ne laissant s'échapper qu'une infime proportion d'entre eux, acceptant paradoxalement d'inclure quelques fausses alertes humaines en collatéral.

Si l'on se place dans le business case typique d'une plateforme de réseaux sociaux qui cherche à éliminer le trafic lié aux bots, il est préférable d'avoir recall élevé quitte à avoir plus de faux positifs : l'humain faux positif peut prouver qu'il n'est pas un robot (captcha, code par e-mail, action manuelle), mais un robot qui passe entre les mailles du filet ne sera jamais débusqué.

6.4. Modélisation Supervisée : SVM

À COMPLÉTER : Analyse de l'application de la théorie du Séparateur à Vaste Marge. Transformation d'échelle, optimisation des noyaux...

7. Modélisation Non-Supervisée

À COMPLÉTER : Analyse exploratoire de détection de groupes k-moyennes / autres algorithmes non-supervisés. Confrontation de la topographie aux observations manuelles annotées.

8. Discussions et Conclusion

À COMPLÉTER : Début de structure de la conclusion. Bilan récapitulatif sur l'hétérogénéité des méthodes adoptées, leurs failles ainsi que leur degré d'opérationnalité.

8.1. Modèle supervisé

COMPLÉTER : choix de l'algorithme (Random Forest, SVM, Logistic Regression, XGBoost...), justification, paramètres optimaux, validation croisée, métriques d'évaluation (accuracy, precision, recall, F1-score, matrice de confusion).

8.2. Modèle non-supervisé

COMPLÉTER : choix de l'algorithme (K-means, DBSCAN, clustering hiérarchique...), justification, choix du nombre de clusters (méthode du coude, silhouette), interprétation des clusters.

8.3. Comparaison des deux approches

COMPLÉTER : méthodologie de comparaison. Comment évaluer le non-supervisé sans labels ? Approches possibles : ARI (Adjusted Rand Index) si on confronte les clusters aux labels manuels, analyse qualitative des clusters, etc.

9. Résultats

SECTION À COMPLÉTER APRÈS AVOIR EXÉCUTÉ LES DEUX MODÈLES.

9.1. Performances du modèle supervisé

COMPLÉTER : présenter la matrice de confusion, le rapport de classification (precision/recall/f1 par classe), la courbe ROC/AUC. Commenter les features les plus importantes (feature importance pour Random Forest, coefficients pour Logistic Regression).

9.2. Résultats du clustering non-supervisé

COMPLÉTER : présenter une visualisation 2D des clusters (t-SNE ou PCA), les centroïdes, la distribution des features par cluster. Interpréter chaque cluster : correspond-il à une classe identifiable ?

9.3. Analyse comparative

COMPLÉTER : comparer les deux approches sur les dimensions suivantes : performance prédictive, interprétabilité, capacité à découvrir des patterns imprévus, dépendance aux labels humains, coût de mise en œuvre.

10. Discussion, limites et perspectives

10.1. Interprétation des résultats

COMPLÉTER : que nous apprennent les résultats ? Les deux approches convergent-elles vers les mêmes profils atypiques ? Y a-t-il des profils détectés par une approche mais pas par l'autre ?

10.2. Limites

COMPLÉTER : discuter les limites de l'étude. Suggestions : taille de l'échantillon labellisé (1 000 profils, peut-être insuffisant) ; biais d'annotation (subjectivité humaine, erreurs de labellisation) ; fenêtre temporelle limitée (Coupe du Monde 2018), les comportements observés sont-ils généralisables ? ; absence de features avancées (analyse de sentiment, NLP sur les tweets, score SPOT complet) ; déséquilibre des classes et impact sur les métriques.

10.3. Perspectives d'industrialisation

COMPLÉTER : comment passer d'un prototype étudiant à un outil utilisable en production ? Suggestions : orchestration du pipeline avec dbt ou Airflow ; mise en place de tests de qualité des données (dbt tests, Great Expectations) ; intégration continue des mises à jour de modèle (MLOps) ; utilisation d'APIs de vérification d'URLs (VirusTotal, Google Safe Browsing) pour enrichir la dimension « danger » du score SPOT ; déploiement du modèle supervisé via une API REST (FastAPI, Flask).

11. Bibliographie

COMPLÉTER : mettre en forme selon le style APA ou IEEE.

- Benevenuto, F., Magno, G., Rodrigues, T., & Almeida, V. (2010). Detecting spammers on Twitter. **Proceedings of the 7th Annual Collaboration, Electronic Messaging, Anti-Abuse and Spam Conference (CEAS)**.
- Chu, Z., Gianvecchio, S., Wang, H., & Jajodia, S. (2012). Detecting automation of Twitter accounts: Are you a human, bot, or cyborg? **IEEE Transactions on Dependable and Secure Computing**, 9(6), 811-824.
- Ferrara, E., Varol, O., Davis, C., Menczer, F., & Flammini, A. (2016). The rise of social bots. **Communications of the ACM**, 59(7), 96-104.
- Perez, C., Lemerrier, M., Birregah, B., & Coppel, A. (2011). SPOT 1.0: Scoring Suspicious Profiles On Twitter. **Proceedings of the 2011 International Conference on Advances in Social Networks Analysis and Mining (ASONAM)**, 377-381.
- Varol, O., Ferrara, E., Davis, C. A., Menczer, F., & Flammini, A. (2017). Online human-bot interactions: Detection, estimation, and characterization. **Proceedings of the 11th International AAAI Conference on Web and Social Media (ICWSM)**, 280-289.

AJOUTER D'AUTRES RÉFÉRENCES PERTINENTES UTILISÉES DANS LE RAPPORT.